



Programming Fundamentals

Week 6 — Working with Data

Programming Fundamentals Team

SETU

2026-09-29





Where Are We Now?

We can now:

- store information using **variables**
- make decisions using **selection**
- organise code using **functions**
- repeat code using **iteration**
- store collections using **lists and dictionaries**



Where Are We Now?

We can now:

- store information using **variables**
- make decisions using **selection**
- organise code using **functions**
- repeat code using **iteration**
- store collections using **lists and dictionaries**

This week is about bringing those ideas together.



From Data Structures to Useful Programs

We already know how to store data:

```
marks = [72, 38, 55, 90, 41]
```

and how to describe one item:

```
student = {"name": "Mary", "mark": 72}
```



From Data Structures to Useful Programs

We already know how to store data:

```
marks = [72, 38, 55, 90, 41]
```

and how to describe one item:

```
student = {"name": "Mary", "mark": 72}
```

The next question is: **what can our program do with the data?**



A Collection of Records

```
students = [  
    {"id": 101, "name": "Mary", "mark": 72},  
    {"id": 102, "name": "John", "mark": 38},  
    {"id": 103, "name": "Sarah", "mark": 55}  
]
```



A Collection of Records

```
students = [  
    {"id": 101, "name": "Mary", "mark": 72},  
    {"id": 102, "name": "John", "mark": 38},  
    {"id": 103, "name": "Sarah", "mark": 55}  
]
```

A **list** stores the collection.

Each **dictionary** represents one student record.



Think About the Data

Given our students collection, what might a useful program need to do?



Think About the Data

Given our students collection, what might a useful program need to do?

- display all students
- find a particular student
- calculate an average
- count passing students
- add a new student
- change a mark
- remove a student



Think About the Data

Given our students collection, what might a useful program need to do?

- display all students
- find a particular student
- calculate an average
- count passing students
- add a new student
- change a mark
- remove a student

These are all examples of **processing data**.



Reading Every Record

Iteration lets us process each record:

```
for student in students:  
    print(student["name"], student["mark"])
```



Reading Every Record

Iteration lets us process each record:

```
for student in students:  
    print(student["name"], student["mark"])
```

Trace it: what does student refer to on each pass through the loop?



Selection Inside Iteration

We can make a decision for every record:

```
for student in students:
    if student["mark"] >= 40:
        print(student["name"], "Pass")
    else:
        print(student["name"], "Fail")
```



Selection Inside Iteration

We can make a decision for every record:

```
for student in students:
    if student["mark"] >= 40:
        print(student["name"], "Pass")
    else:
        print(student["name"], "Fail")
```

Iteration chooses **which record**. Selection chooses **what to do with it**.



Processing Pattern: Count

```
passes = 0

for student in students:
    if student["mark"] >= 40:
        passes = passes + 1

print("Number of passes:", passes)
```




Processing Pattern: Count

```
passes = 0

for student in students:
    if student["mark"] >= 40:
        passes = passes + 1

print("Number of passes:", passes)
```

We have seen this pattern before — the difference is that our data is now structured.



Processing Pattern: Total and Average

```
total = 0

for student in students:
    total = total + student["mark"]

average = total / len(students)
print("Average:", average)
```



Processing Pattern: Total and Average

```
total = 0

for student in students:
    total = total + student["mark"]

average = total / len(students)
print("Average:", average)
```

What happens if students is empty?



Processing Pattern: Search

```
search_id = int(input("Student ID: "))
found = False

for student in students:
    if student["id"] == search_id:
        print(student)
        found = True

if found == False:
    print("Student not found")
```



Processing Pattern: Search

```
search_id = int(input("Student ID: "))  
found = False
```

```
for student in students:  
    if student["id"] == search_id:  
        print(student)  
        found = True
```

```
if found == False:  
    print("Student not found")
```

Why is the found variable useful?



Put the Search in a Function

```
def find_student(students, search_id):  
    for student in students:  
        if student["id"] == search_id:  
            return student  
  
    return None
```



Put the Search in a Function

```
def find_student(students, search_id):  
    for student in students:  
        if student["id"] == search_id:  
            return student  
  
    return None  
  
student = find_student(students, 102)
```

The function has one job: find and return a student.



Why Return the Student?

Compare:

```
def find_student(students, search_id):  
    # ...  
    print(student)
```

with:

```
def find_student(students, search_id):  
    # ...  
    return student
```




Why Return the Student?

Compare:

```
def find_student(students, search_id):  
    # ...  
    print(student)
```

with:

```
def find_student(students, search_id):  
    # ...  
    return student
```

Returning the record means the rest of our program can decide what to do with it.



A New Way to Think About Our Program

Instead of one long script, we can give functions responsibilities:

```
display_students()  
find_student()  
calculate_average()  
count_passes()
```



A New Way to Think About Our Program

Instead of one long script, we can give functions responsibilities:

```
display_students()  
find_student()  
calculate_average()  
count_passes()
```

Functions organise the **operations**. Lists and dictionaries organise the **data**.



Now We Want to Change the Data

So far we have mostly **read** and processed existing data.

What if the user needs to:

- add a student?
- change a student's mark?
- remove a student?



Now We Want to Change the Data

So far we have mostly **read** and processed existing data.

What if the user needs to:

- add a student?
- change a student's mark?
- remove a student?

These operations have a very common name.



CRUD

CRUD describes four basic operations used in data-driven programs:

Operation	Meaning
Create	Add new data
Read	View or find data
Update	Change existing data
Delete	Remove data



CRUD Is Already Familiar

Think about an app you use regularly.

A contacts app might let you:

- **Create** a contact
- **Read** your contacts
- **Update** a phone number
- **Delete** a contact



CRUD Is Already Familiar

Think about an app you use regularly.

A contacts app might let you:

- **Create** a contact
- **Read** your contacts
- **Update** a phone number
- **Delete** a contact

The terminology is new. The ideas are not.



Mini Activity — Design the Operations

Imagine a simple **Dog Day Care Manager**.

Each dog has:

id, name, breed, age

For each CRUD operation, write down:

- what information the user needs to provide
- what the program needs to find or change
- what could go wrong



Mini Activity — Design the Operations

Imagine a simple **Dog Day Care Manager**.

Each dog has:

id, name, breed, age

For each CRUD operation, write down:

- what information the user needs to provide
- what the program needs to find or change
- what could go wrong

We will use exactly this thinking when we build CRUD code.



Lecture 1 — Key Ideas

- Lists can hold collections of records
- Dictionaries can represent individual records
- Loops let us process every record
- Selection lets us decide what to do with each record
- Functions give operations clear responsibilities
- CRUD means **Create, Read, Update, Delete**



Lecture 1 — Key Ideas

- Lists can hold collections of records
- Dictionaries can represent individual records
- Loops let us process every record
- Selection lets us decide what to do with each record
- Functions give operations clear responsibilities
- CRUD means **Create, Read, Update, Delete**

Next: we turn these ideas into a small working CRUD application.